AI FinOps: 토큰 시대의 비용 관리

생성형 AI를 처음 도입할 때는 보통 “무엇을 만들 수 있을까”부터 이야기한다. 사내 챗봇, 문서 요약, 코딩 보조, 고객 문의 자동화처럼 눈에 보이는 사용 사례를 찾는 단계다.

그런데 몇 개월 지나 실제 업무와 제품에 AI가 붙기 시작하면 회의의 주제가 달라진다. 이번 달 API 비용은 왜 늘었는지, 특정 팀의 사용량은 왜 튀었는지, 이 기능이 매달 드는 비용만큼 효과를 내고 있는지 따지게 된다.

AI FinOps는 여기서 출발한다. LLM API 비용을 조금 아끼는 활동만을 뜻하지 않는다. 토큰, GPU, 모델 품질, 응답 속도, 데이터 보관, 보안 요구사항, 그리고 실제 업무 성과를 같이 보면서 AI 비용을 설명 가능한 상태로 만드는 일에 가깝다.

AI 비용은 어디서 생기나

기존 클라우드 비용은 비교적 익숙하다. 서버를 얼마나 띄웠는지, 스토리지를 얼마나 썼는지, 네트워크 트래픽이 얼마나 나갔는지를 보면 된다.

AI 비용도 큰 틀에서는 같다. 결국 **가격과 사용량의 곱**이다. 다만 사용량을 재는 단위가 훨씬 많아졌다. 입력 토큰, 출력 토큰, 캐시 토큰, API 호출 수, GPU 사용 시간, 벡터 DB 저장량, 임베딩 생성량, 파인튜닝 비용, 모델 배포 인스턴스가 모두 비용 항목이 된다.

특히 토큰 비용은 예측이 어렵다. 같은 기능이라도 사용자가 긴 질문을 하거나, RAG가 많은 문서를 가져오거나, 모델이 긴 답변을 만들면 비용이 달라진다. 에이전트 구조에서는 한 번의 사용자 요청 안에서 여러 번 추론하고 도구를 호출하기 때문에 비용이 더 잘게 쪼개진다.

그래서 AI FinOps의 첫 단계는 비용을 줄이는 것이 아니다. 먼저 어디서 비용이 나가는지 볼 수 있어야 한다.

먼저 기록해야 할 것들

AI 비용을 제대로 보려면 최소한 다음 정도는 기록해야 한다.

- 팀, 제품, 기능, 사용자 또는 테넌트별 사용량
- 모델별 입력 토큰, 출력 토큰, 캐시 토큰
- 요청 1건당 비용
- 작업 완료 1건당 비용
- 응답 시간, 실패율, 재시도율
- RAG 검색 문서 수와 컨텍스트 길이
- 에이전트의 단계 수와 도구 호출 수
- 자동 해결률, 상담 시간 절감, 전환율 같은 업무 지표

이 숫자들이 연결되어야 “비싼 기능”과 “돈값을 하는 기능”을 구분할 수 있다. 어떤 기능은 비용이 높아도 상담 시간을 크게 줄인다면 유지할 만하다. 반대로 비용은 작아 보여도 아무도 쓰지 않거나 품질이 낮다면 정리하는 편이 낫다.

토큰 비용을 줄이는 방법

가장 먼저 손볼 수 있는 것은 토큰이다. 다만 프롬프트를 무조건 짧게 만든다고 좋은 것은 아니다. 필요한 맥락까지 줄이면 답변 품질이 떨어지고, 사용자가 다시 질문하면서 총비용이 오히려 늘 수 있다.

먼저 출력 길이를 제한해야 한다. 많은 경우 출력 토큰이 비용에 큰 영향을 준다. 요약은 몇 문장까지 허용할지, JSON 응답은 어떤 필드만 줄지, 답변 형식은 어떻게 고정할지를 정해두는 편이 좋다.

반복되는 프롬프트도 정리해야 한다. 모든 요청마다 긴 정책 문서, 예시, 도구 설명을 통째로 보내면 비용이 계속 쌓인다. 공통 지시문과 사용자별 입력을 분리하고, 자주 반복되는 내용은 캐시가 잘 먹도록 프롬프트 앞부분에 안정적으로 배치한다.

RAG도 비용 관점에서 다시 봐야 한다. 검색 결과를 많이 넣는다고 답변이 항상 좋아지는 것은 아니다. 중복 문서를 제거하고, reranking을 거치고, 필요한 부분만 잘라 넣는 편이 낫다. 긴 컨텍스트는 비용뿐 아니라 정확도에도 영향을 줄 수 있다.

에이전트에는 예산을 둬야 한다. 최대 단계 수, 최대 도구 호출 수, 최대 실행 시간, 최대 토큰 수를 정하지 않으면 작은 요청 하나가 예상보다 큰 비용으로 커질 수 있다.

캐싱은 가장 먼저 확인할 절감 수단

AI 비용을 줄일 때 캐싱은 먼저 확인할 만한 방법이다. OpenAI, Anthropic, Google Gemini, Amazon Bedrock 등 주요 서비스가 반복되는 프롬프트나 긴 컨텍스트를 재사용하는 기능을 제공한다.

핵심은 프롬프트 구조다. 시스템 프롬프트, 도구 정의, 긴 정책 문서, 공통 예시처럼 자주 반복되는 내용은 앞에 둔다. 사용자 질문, 날짜, 개인화 정보처럼 매번 바뀌는 내용은 뒤에 둔다. 그래야 같은 prefix가 반복되고 캐시 적중률이 올라간다.

고객센터 FAQ나 사내 지식 검색처럼 비슷한 질문이 반복되는 서비스라면 semantic cache도 검토할 수 있다. 완전히 같은 문장이 아니더라도 의미가 비슷한 질문에 이전 답변을 재사용하는 방식이다. 다만 오래된 답변이나 문맥이 다른 답변이 재사용되지 않도록 만료 정책과 유사도 기준을 조심해서 잡아야 한다.

실시간 답변이 필요 없는 일은 Batch로 보낸다

모든 AI 작업이 즉시 끝나야 하는 것은 아니다. 문서 분류, 대량 요약, 임베딩 생성, 평가셋 실행, 콘텐츠 태깅, 로그 분석은 비동기 처리에 잘 맞는다.

이런 작업은 Batch API를 쓰면 비용과 처리량 면에서 유리할 수 있다. 대신 즉시 결과를 받을 수 없고, 실패한 요청을 다시 처리하는 로직이 필요하다. 사용자 화면에서 바로 답해야 하는 기능보다는 백오피스 작업, 야간 처리, 데이터 파이프라인, 평가 자동화에 먼저 적용하는 편이 현실적이다.

비싼 모델을 기본값으로 두지 않는다

모든 요청을 가장 비싼 모델로 보낼 필요는 없다. 짧은 분류, 형식 변환, 키워드 추출, 간단한 요약은 더 작은 모델로도 충분한 경우가 많다.

실무에서는 보통 이런 구조가 유용하다. 기본 요청은 저렴하고 빠른 모델로 처리한다. 답변 품질이 낮거나 확신도가 낮은 경우에만 상위 모델로 넘긴다. 법무, 보안, 의료, 금융처럼 위험도가 높은 작업은 처음부터 더 신중한 모델과 검증 절차를 붙인다.

중요한 것은 평가셋이다. 비용만 보고 모델을 낮추면 품질 저하를 놓치기 쉽다. 정확도, 재작업률, 사용자 만족도, 사람에게 넘어가는 비율을 같이 봐야 한다.

GPU 비용은 유휴 시간을 줄이는 것이 먼저다

자체 모델을 운영하거나 클라우드 GPU를 쓰는 조직은 API 토큰 비용만 보면 안 된다. GPU는 켜져 있는 시간 자체가 비용이다. 사용률이 낮은 GPU, 방치된 실험 환경, 오래된 모델 배포, 불필요하게 큰 인스턴스가 비용을 만든다.

GPU 비용을 보려면 모델별 처리량, 응답 시간, GPU/메모리 사용률, batch 처리 가능 여부, autoscaling 정책을 함께 봐야 한다. 예약, 온디맨드, 스팟을 어떻게 섞을지도 사용 패턴에 따라 달라진다.

Quantization, distillation, speculative decoding, batch inference 같은 기법도 비용을 줄이는 데 도움이 된다. 다만 이런 기법은 품질과 지연시간에 영향을 줄 수 있으므로 실제 트래픽과 평가셋으로 확인해야 한다.

단계별로 시작하기

처음부터 완벽한 AI 비용 관리 체계를 만들 필요는 없다.

초기에는 실험별 예산과 기간을 정하고, 어떤 모델을 얼마나 썼는지만 기록해도 충분하다. 중요한 것은 작은 실험을 작게 끝내는 것이다.

그다음에는 팀별 사용량, 기능별 비용, 모델별 비용을 대시보드로 보여준다. 출력 길이 제한, 캐싱, Batch 처리처럼 위험이 낮은 절감 방법부터 적용한다.

AI 사용이 핵심 업무에 들어가기 시작하면 자동 알림, 이상 사용 탐지, 모델 라우팅, GPU 약정 구매, ROI 기준의 우선순위 결정까지 필요해진다. 이 단계에서는 비용 절감보다 어떤 AI 사용에 더 투자할지를 판단하는 일이 더 중요해진다.

마무리

AI FinOps의 목적은 AI 사용을 막는 것이 아니다. 좋은 사용 사례는 더 많이 쓰이게 해야 한다. 대신 반복 프롬프트, 과도한 컨텍스트, 불필요하게 긴 출력, 제한 없는 에이전트 루프, 놀고 있는 GPU, 검증 없이 쓰는 고가 모델은 줄여야 한다.

앞으로 AI 비용 관리는 단순한 클라우드 비용 관리 항목으로 남지 않을 가능성이 높다. 제품팀, 엔지니어링 팀, 재무팀이 같은 숫자를 보면서 “이 AI 기능이 비용만큼의 일을 하고 있는가”를 계속 확인해야 한다.

결국 중요한 질문은 하나다. 우리는 토큰을 얼마나 썼는가가 아니라, 그 토큰으로 어떤 비즈니스 결과를 만들었는가를 묻게 될 것이다.

참고자료

- FinOps Foundation, FinOps for AI Overview
- OpenAI, Prompt Caching
- OpenAI, Batch API
- Anthropic, Prompt caching
- Anthropic, Batch processing
- Google Cloud, Context caching overview
- AWS, Prompt caching for faster model inference - Amazon Bedrock
- WIRED, ‘Pretty Crazy’ Token Usage Is Testing Bosses’ Bet on AI
- ITPro, Could rising token costs boost interest in on-premises hardware?
- arXiv, Prompt Cache: Modular Attention Reuse for Low-Latency Inference
- arXiv, GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching
- arXiv, Towards Cost-effective LLMs Routing with Batch Prompting

4